Artificial Intelligence: Assignment 1

Using Informed and Uninformed Search Algorithms to Solve 8-Puzzle

Ali Amr Abdullah Ahmed (23010589)
Nour Eldeen Mohammed Abbas (23010920)

1 Overview & Problem Formulation

The 8-puzzle problem consists of a 3×3 board containing eight numbered movable tiles (1 through 8) and one blank space represented by 0. Given an initial configuration, the objective is to find a sequence of legal actions (`Up`, `Down`, `Left`, `Right`) that transitions the board to the goal state $(0, 1, 2, 3, 4, 5, 6, 7, 8)$ with uniform step cost equal to 1.

2 Data Structures & Architecture

To guarantee optimal time and space complexity across all search strategies, the following core data structures were implemented:

- **Board State Representation (`tuple[int, ...]`):** Each board state is stored as an immutable 1D tuple of 9 integers rather than a 2D list. Because Python tuples are hashable, membership checks in the `explored` set and `frontier` dictionaries execute in $O(1)$ average time instead of $O(N)$ linear scans.
- **Search Tree Node (`PuzzleNode`):** Each node encapsulates `state`, `parent` pointer, `action`, current depth $g(n)$, heuristic estimate $h(n)$, and a sequential `node_id`. Rather than copying the entire board history at each step, the solution path is reconstructed upon reaching the goal by traversing `parent` pointers backward to the root in $O(d)$ time.
- **Algorithm-Specific Frontier Structures:**
  - **BFS:** Uses `collections.deque` as a FIFO queue alongside a `reached` hash set (frontier $\cup$ explored).
  - **DFS:** Uses a Python `list` as a LIFO stack alongside a `reached` hash set.
  - **Iterative DFS (IDDFS):** Uses a LIFO stack paired with a `best_depth` hash map (`dict[BoardState, int]`) per depth iteration.
  - **A* Search:** Uses a binary min-heap priority queue (`heapq`) ordered by $f(n) = g(n) + h(n)$, paired with an `explored` set and a `best_g` dictionary.

3 Search Algorithms & Operation

3.1 Breadth-First Search (BFS)

BFS expands the shallowest unexpanded nodes first using a FIFO queue. Because all step costs are equal to 1, BFS is both **complete** and **optimal** (always finds the shortest path of cost d), with time and space complexity $O(b^d)$.

3.2 Depth-First Search (DFS)

DFS expands the deepest node in the frontier using a LIFO stack. To preserve the canonical action evaluation order (`Up`, `Down`, `Left`, `Right`), neighbor states are pushed onto the stack in reverse order. While DFS is complete on the finite 8-puzzle state space when cycle-checking is enabled, it is **not optimal** and frequently returns paths thousands of moves long.

3.3 Iterative Deepening DFS (IDDFS)

IDDFS combines the $O(bd)$ linear memory footprint of DFS with the optimality of BFS by repeatedly executing Depth-Limited Search (DLS) for depth limits $L = 0, 1, 2, \ldots, d$. Within each iteration, a state is only generated if it has not been visited in the current iteration or if it is reached at a strictly shallower depth ($g_{\text{new}} <$ best_depth[state]), preventing suboptimal deep branches from blocking shorter paths to the same state.

3.4 Informed Search: A*

A* evaluates nodes using $f(n) = g(n) + h(n)$, where $g(n)$ is the exact path cost from the start node and $h(n)$ is the estimated cost to the goal. When $f(n)$ values tie, the priority queue breaks ties deterministically by smallest $h(n)$, followed by earliest creation order (`node_id`).

4 Heuristics Comparison & Admissibility Analysis

For a tile located at coordinates (x_c, y_c) with target goal coordinates (x_g, y_g) (excluding the blank tile 0), we implemented two heuristics:

1. **Manhattan Distance:**

$$h_{\text{Manhattan}}(n) = \sum_{i=1}^{8} \left(|x_{c,i} - x_{g,i}| + |y_{c,i} - y_{g,i}| \right)$$

2. **Euclidean Distance:**

$$h_{\text{Euclidean}}(n) = \sum_{i=1}^{8} \sqrt{(x_{c,i} - x_{g,i})^2 + (y_{c,i} - y_{g,i})^2}$$

Which Heuristic is More Admissible / Informed? A heuristic is **admissible** if it never overestimates the true minimum cost $h^*(n)$ to reach the goal ($h(n) \leq h^*(n)$). Because legal moves in the 8-puzzle are strictly horizontal or vertical (no diagonal movement), every tile must move at least $|x_c - x_g| + |y_c - y_g|$ steps. By the triangle inequality, for all non-negative coordinate differences $\Delta x, \Delta y$:

$$\sqrt{\Delta x^2 + \Delta y^2} \leq \Delta x + \Delta y \leq h^*(n)$$

Thus, $0 \leq h_{\text{Euclidean}}(n) \leq h_{\text{Manhattan}}(n) \leq h^*(n)$. Both heuristics are strictly admissible and guarantee optimal shortest paths. However, **Manhattan distance dominates (is more informed than) Euclidean distance** because it provides a tighter lower bound closer to the true cost $h^*(n)$. Consequently, A* with Manhattan distance consistently expands fewer or equal nodes compared to Euclidean distance.

5 Assumptions & Extra Work

- **Solvability Verification (Inversion Parity):** Half of all $9! = 362,880$ board configurations are mathematically unsolvable. Because the goal state $(0, 1, 2, 3, 4, 5, 6, 7, 8)$ has 0 inversions and legal orthogonal swaps on a 3×3 grid preserve inversion parity, our solver verifies that the number of inversions is even in $O(N^2)$ prior to starting any search.
- **Goal Test Placement:** The goal test is applied when a node is popped from the frontier, and the goal node itself is not counted in `nodes_expanded`.
- **Extra Work – Interactive PySide6 Search Tree Visualizer:** We engineered a full desktop GUI using `PySide6` featuring:
  1. An interactive setup screen with live solvability validation and preset/random board generation.
  2. A dynamic `QGraphicsView` search tree canvas with recursive subtree-width layout, pan/zoom navigation, and step-by-step timeline playback (`Play`, `Pause`, `Step +/-`, speed control, and timeline scrubber).
  3. Color-coded node states (*Frontier*, *Currently Expanding*, *Visited*, and *Goal Path*) with a right-hand **Node Inspector** dock and **Goal Path Trace** viewer.

6 Sample Runs

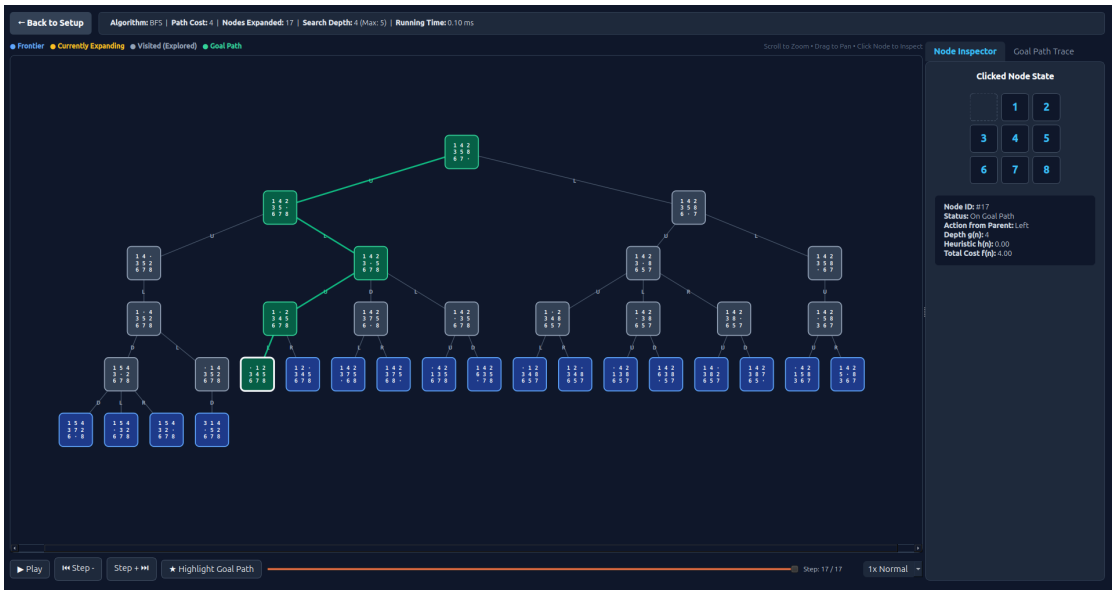


Figure 1: Sample Run 1: Breadth-First Search (BFS) showing the highlighted goal path, metrics bar, and step-by-step Goal Path Trace.



Figure 2: Sample Run 2: Depth-First Search (DFS) execution and metrics.

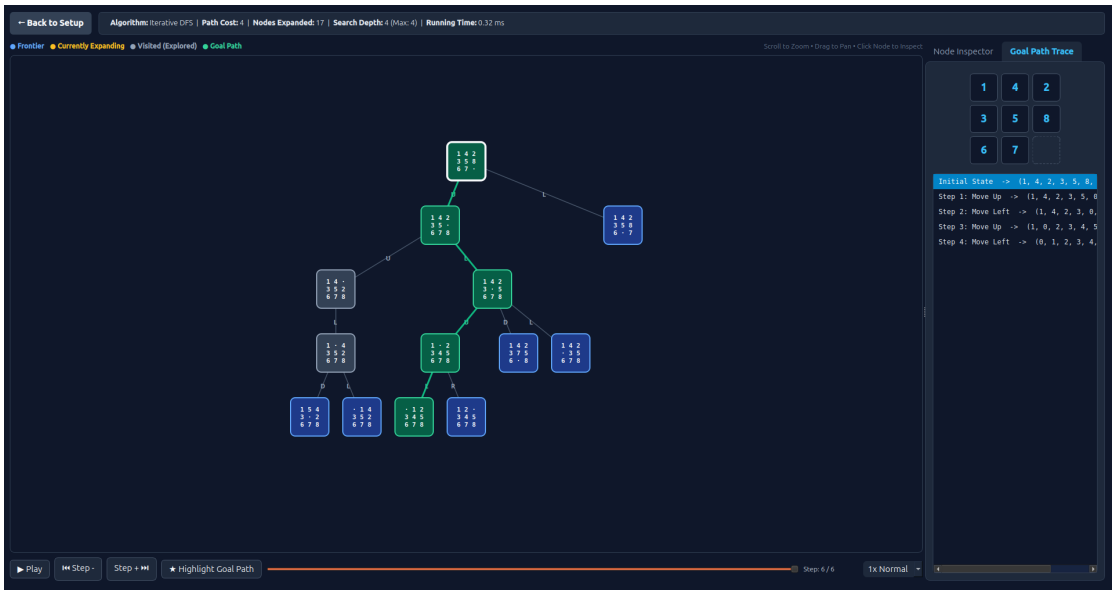


Figure 3: Sample Run 3: Iterative Deepening DFS (IDDFS) execution and final depth-limit tree.

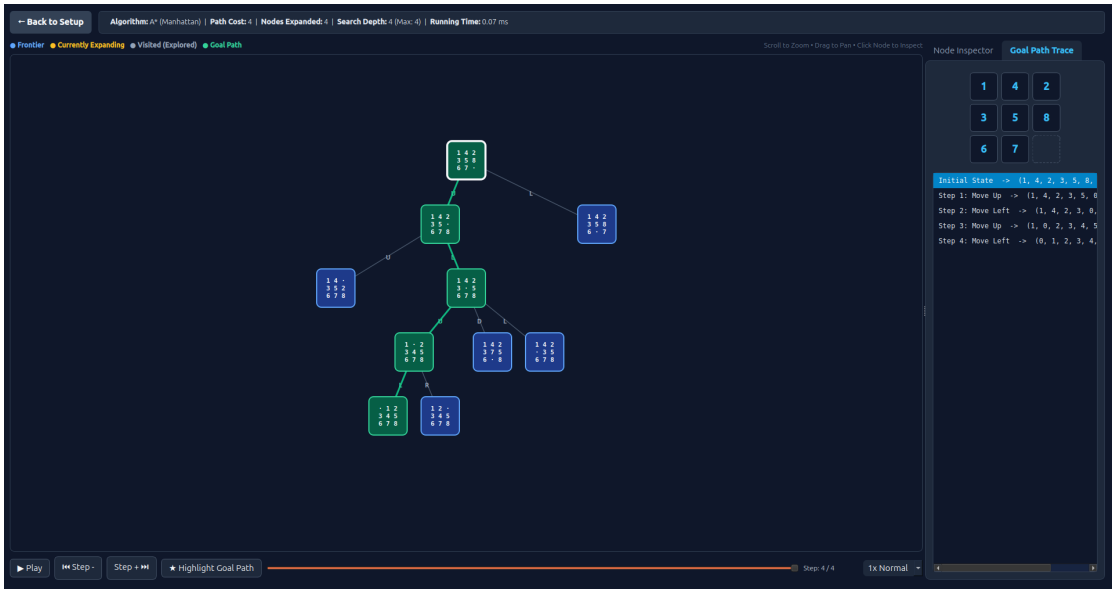


Figure 4: Sample Run 4: A* Search (Manhattan Heuristic) expanding only the optimal path nodes.